

컴퓨터 비전

동양미래대학교
인공지능소프트웨어학과
백찬은 교수

컴퓨터 비전의 인식 문제

01

컴퓨터 비전의 인식 문제

02

객체 검출 문제에서의 성능 측정

03

객체 검출 실습

04

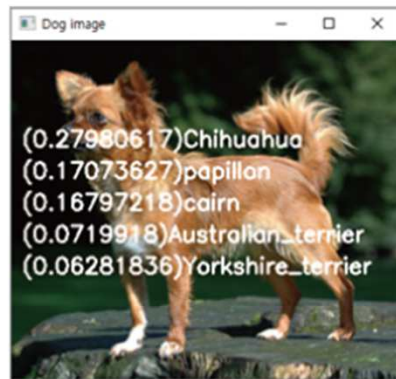
분할 실습



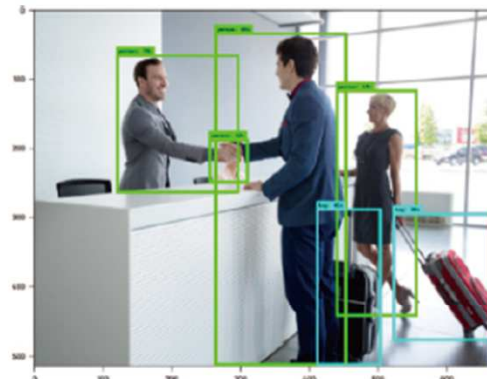


컴퓨터 비전의 인식 문제

여러가지 인식 문제



(a) 분류



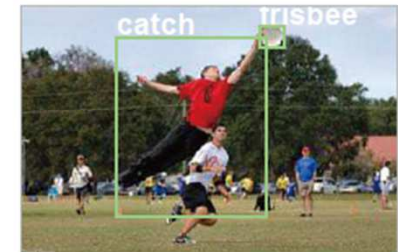
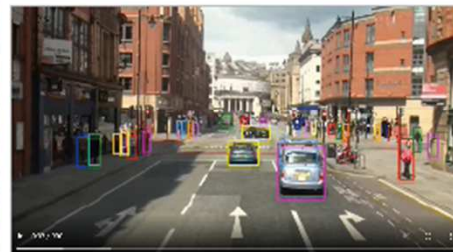
(b) 검출



(c) 분할



(d) 추적(0초, 3초, 6초 순간)



(e) 행동 분류

그림 9-2 인식을 여러 세부 문제로 나누는 컴퓨터 비전

여러가지 인식 문제

- **분류(classification)**

- 영상에 있는 물체가 무엇인지 알아내는 문제.
- 한 장의 이미지에 대해 하나의 라벨을 예측함.
- 출력: 클래스 확률 벡터
- 예: COCO 데이터셋 - 사람, 자전거, 자동차 등 80개의 클래스에 속할 각각의 확률 벡터를 출력함
- **종류: 범주 분류, 사례 분류**
- **범주 분류(Categorical Classification)**
 - 가장 기본적인 이미지 분류 문제
 - 고양이, 개, 자동차처럼 클래스 자체를 예측함
- **사례 분류(Instance Classification)**
 - crop된 물체의 이미지를 보고, 그 물체가 어떤 class에 속하는지 판별하는 문제
 - 보통 객체 검출/세분화에서 각 인스턴스에 클래스 라벨을 붙이는 단계에서 사용됨

여러가지 인식 문제

- **검출(detection)**
 - 영상에서 물체를 찾아 직사각형(바운딩 박스)으로 위치 표현
 - 보통 class에 속할 확률도 같이 출력함.
- **분할(segmentation)**
 - 물체가 점유하는 화소를 판별하는 문제. class에 속할 확률도 함께 출력함
 - **종류: Semantic segmentation, Instance Segmentation**
 - **Semantic Segmentation**
 - 같은 클래스에 속하는 모든 픽셀을 하나의 그룹으로 분류함
 - (예) 모든 '사람' 픽셀을 동일하게 '사람' 클래스로 처리함
 - **Instance Segmentation**
 - 같은 클래스에 속하더라도 서로 다른 물체라면, 서로 다른 instance로 분리함
 - (예) 사람이 3명 있으면, 사람1, 사람2, 사람3으로 따로 구분함

여러가지 인식 문제

- **추적(tracking)**
 - 영상(비디오)에서 특정 물체가 시간에 따라 어떻게 이동하는지 연속적으로 추적하는 문제
 - **종류: VOT, MOT**
 - **VOT(Visual Object Tracking)**
 - 단일 물체를 추적하는 문제
 - 영상 속에 등장한 하나의 객체를 안정적으로 계속 추적함
 - (예) 특정 선수, 특정 자동차를 추적
 - **MOT(Multiple Object Tracking)**
 - 다중 물체를 동시에 추적하는 문제
 - 객체마다 ID(고유 번호)를 유지하면서 움직임을 추적함
 - (예) 군중 추적, 여러 차량 추적, CCTV 보행자 추적

여러가지 인식 문제

- **행동 분류(Action Recognition)**
 - 비디오 속 물체(주로 사람)가 어떤 행동을 하고 있는지 분류하는 문제
 - (예) 스포츠 분석, CCTV 이상행동 감지



객체 검출 문제에서의 성능 측정

IoU(Intersection over Union)

- IoU(Intersection over Union)
- : 객체 검출에서 정답 박스와 예측한 박스가 얼마나 겹쳤는지를 수치로 표현한 값
- $IoU = (\text{두 박스가 동시에 겹친 영역의 면적}) / (\text{두 박스가 합쳐진 영역의 면적})$
- 일반적으로 정답으로 인정되는 최소 기준으로 IoU 0.5이상 또는 0.75이상을 택함

$$IoU = \frac{area(B_{predict} \cap B_{GT})}{area(B_{predict} \cup B_{GT})} \quad (9.1)$$

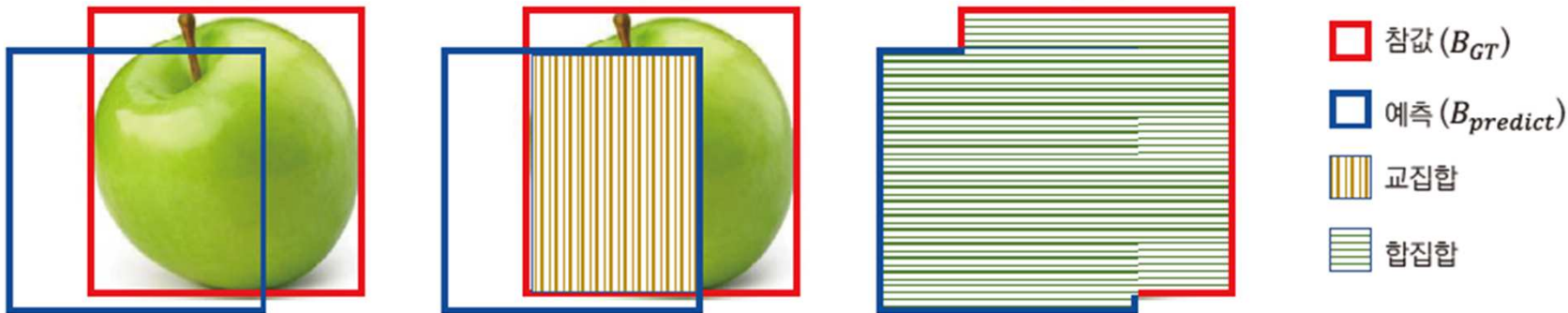
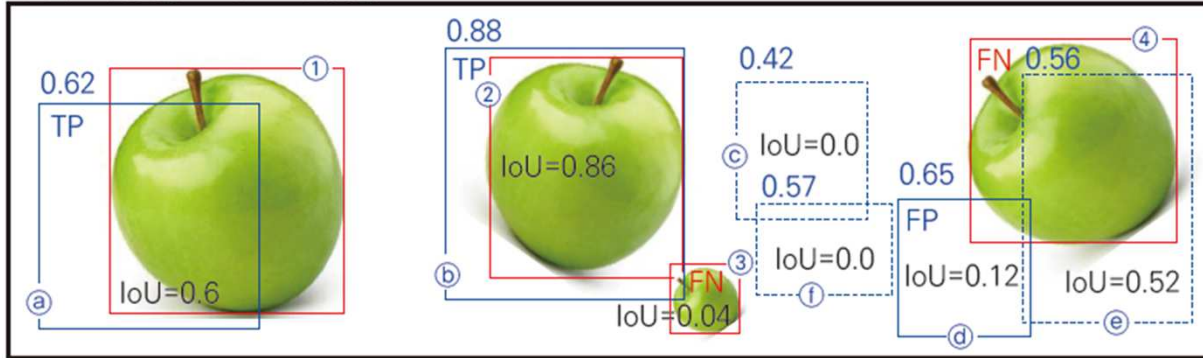


그림 9-14 IoU 계산

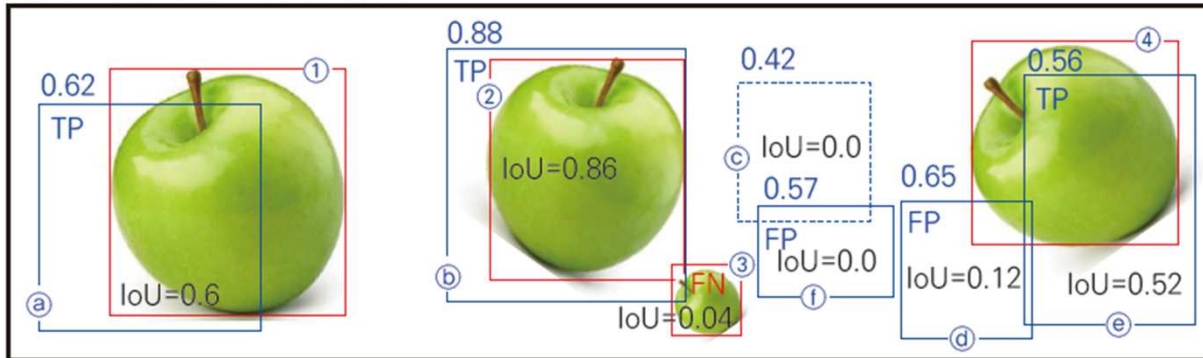
IoU(Intersection over Union)

신뢰도 임계값=0.6, IoU 임계값=0.5



(a) 신뢰도 임계값을 0.6으로 설정

신뢰도 임계값=0.5, IoU 임계값=0.5



(b) 신뢰도 임계값을 0.5로 설정

그림 9-15 TP, FP, FN 세기(IoU 임계값=0.5)

①~④는 참값 박스
a~f는 예측 박스

TP / FP / FN 판정하기

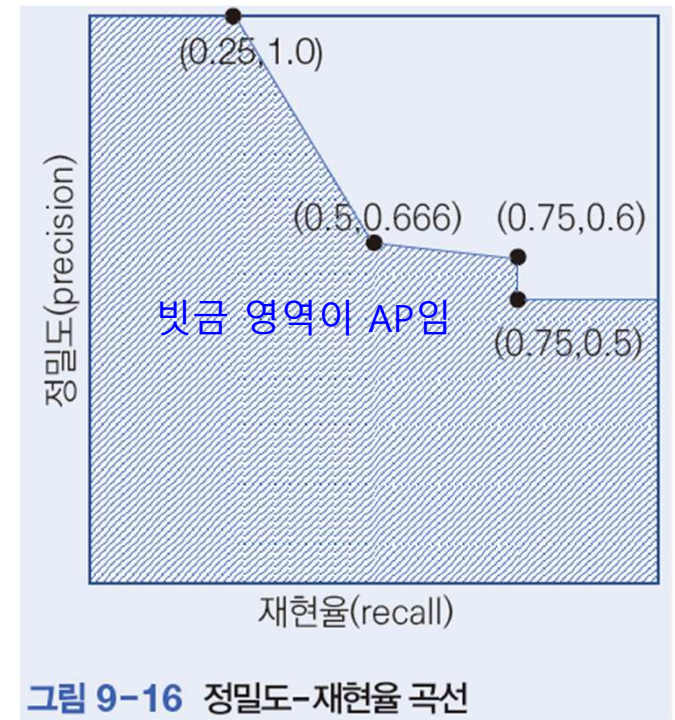
- 예측 박스가 여러 개 있을 때, "정확히 맞췄는가?"를 판단해야 함
- 판단 규칙
 1. 예측 박스를 신뢰도(score) 순으로 정렬함
 2. 위에서부터 하나씩 정답 박스와 비교함
 3. IoU가 임계값보다 크거나 같으면 TP
 - 해당 정답 박스는 더이상 다른 물체와 매칭되지 않음
 4. IoU가 임계값보다 작으면 FP
 5. 예측되지 못한(매칭되지 못한) 정답 박스는 FN

Precision과 Recall 계산

- 신뢰도 임계값을 바꿔가며 Precision, Recall 계산하여 PR 곡선을 생성함
- 신뢰도 임계값을 0.0, 0.1, 0.2, ... , 1.0까지 바꿔가며
- 정밀도(Precision) = $TP / (TP+FP)$
- 재현율(Recall) = $TP / (TP+FN)$
- 임계값을 낮추면 더 많은 박스를 포함하고(Recall 증가, Precision 감소)
- 임계값을 높이면 더 엄격하게 박스를 선택함(Recall 감소, Precision 증가)
- 이를 바탕으로 정밀도-재현율 곡선을 생성함(PR Curve)

PR 곡선 생성 및 AP(Average Precision) 계산

- 앞서 구한 (Recall, Precision) 점들을 좌표에 찍어 연결한 그래프가 PR 곡선임
 - 가로축 Recall, 세로축 Precision
- 이 곡선 아래 면적(Area Under Curve)이 바로 AP(Average Precision)가 됨
- 모든 class에 대해 AP를 구한 후, 평균을 내면 mAP(mean Average Precision)가 됨



성능 측정

- mAP 값으로서 성능을 측정함
- **mAP(mean Average Precision) 구하는 과정**
- 1단계: 예측한 박스 필터링
- 2단계: 예측한 박스를 신뢰도 높은 순서로 정렬함
- 3단계: IoU를 기준으로 TP/FP/FN 판정
- 4단계: 신뢰도 임계값을 0.0부터 1.0까지 변화시키며 PR curve 작성
- 5단계: 각 클래스의 AP 계산
- 6단계: 데이터셋 내의 모든 클래스에 대한 mAP 계산



객체 검출 실습

(실습) YOLO v3으로 정지 영상에서 물체 검출하기

프로그램 9-1

YOLO v3으로 정지 영상에서 물체 검출하기

```
01 import numpy as np
02 import cv2 as cv          사전 학습 모델을 읽어 YOLO 구성
03 import sys
04
05 def construct_yolo_v3():
06     f=open('coco_names.txt', 'r')
07     class_names=[line.strip() for line in f.readlines()]
08
09     model=cv.dnn.readNet('yolov3.weights','yolov3.cfg')
10     layer_names=model.getLayerNames()
11     out_layers=[layer_names[i-1] for i in model.getUnconnectedOutLayers()]
12
13     return model,out_layers,class_names
14
```

(실습) YOLO v3으로 정지 영상에서 물체 검출하기

```
15 def yolo_detect(img,yolo_model,out_layers): ← YOLO 모델로 물체를 검출
16     height,width=img.shape[0],img.shape[1]
17     test_img=cv.dnn.blobFromImage(img,1.0/256,(448,448),(0,0,0),swapRB=True)
18
19     yolo_model.setInput(test_img)
20     output3=yolo_model.forward(out_layers)
21
22     box,conf,id=[],[],[] # 박스, 신뢰도, 부류 번호
23     for output in output3:
24         for vec85 in output:
25             scores=vec85[5:]
26             class_id=np.argmax(scores)
27             confidence=scores[class_id]
28             if confidence>0.5: # 신뢰도가 50% 이상인 경우만 취함
29                 centerx,centery=int(vec85[0]*width),int(vec85[1]*height)
30                 w,h=int(vec85[2]*width),int(vec85[3]*height)
31                 x,y=int(centerx-w/2),int(centery-h/2)
32                 box.append([x,y,x+w,y+h])
33                 conf.append(float(confidence))
34                 id.append(class_id)
35
36     ind=cv.dnn.NMSBoxes(box,conf,0.5,0.4)
37     objects=[box[i]+[conf[i]]+[id[i]] for i in range(len(box)) if i in ind]
38     return objects
```

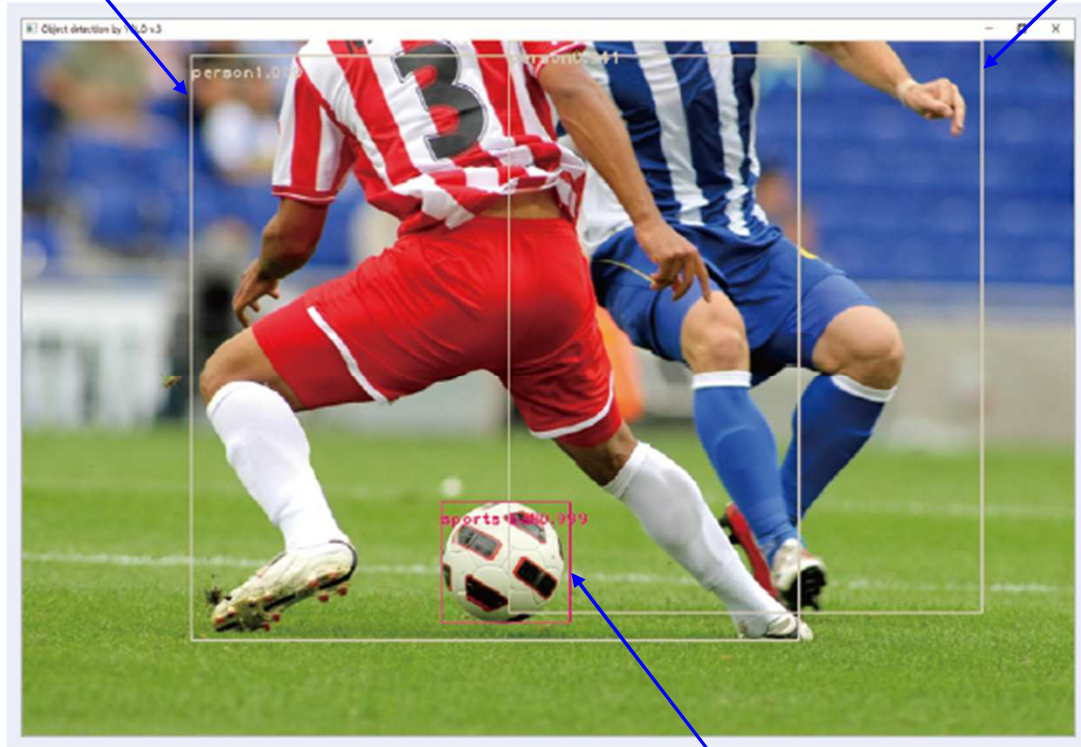
(실습) YOLO v3으로 정지 영상에서 물체 검출하기

```
39
40 model,out_layers,class_names=construct_yolo_v3()           # YOLO 모델 생성
41 colors=np.random.uniform(0,255,size=(len(class_names),3)) # 부류마다 색깔
42
43 img=cv.imread('soccer.jpg')
44 if img is None: sys.exit('파일이 없습니다.')
45
46 res=yolo_detect(img,model,out_layers)                       # YOLO 모델로 물체 검출
47
48 for i in range(len(res)):                                  # 검출된 물체를 영상에 표시
49     x1,y1,x2,y2,confidence,id=res[i]
50     text=str(class_names[id])+'.3f'%confidence
51     cv.rectangle(img,(x1,y1),(x2,y2),colors[id],2)
52     cv.putText(img,text,(x1,y1+30),cv.FONT_HERSHEY_PLAIN,1.5,colors[id],2)
53
54 cv.imshow("Object detection by YOLO v.3",img)
55
56 cv.waitKey()
57 cv.destroyAllWindows()
```

(실습) YOLO v3으로 정지 영상에서 물체 검출하기

1.00 신뢰도로 person 부류

0.941 신뢰도로 person 부류



0.999 신뢰도로 sports ball 부류

(실습) YOLO v3으로 정지 영상에서 물체 검출하기

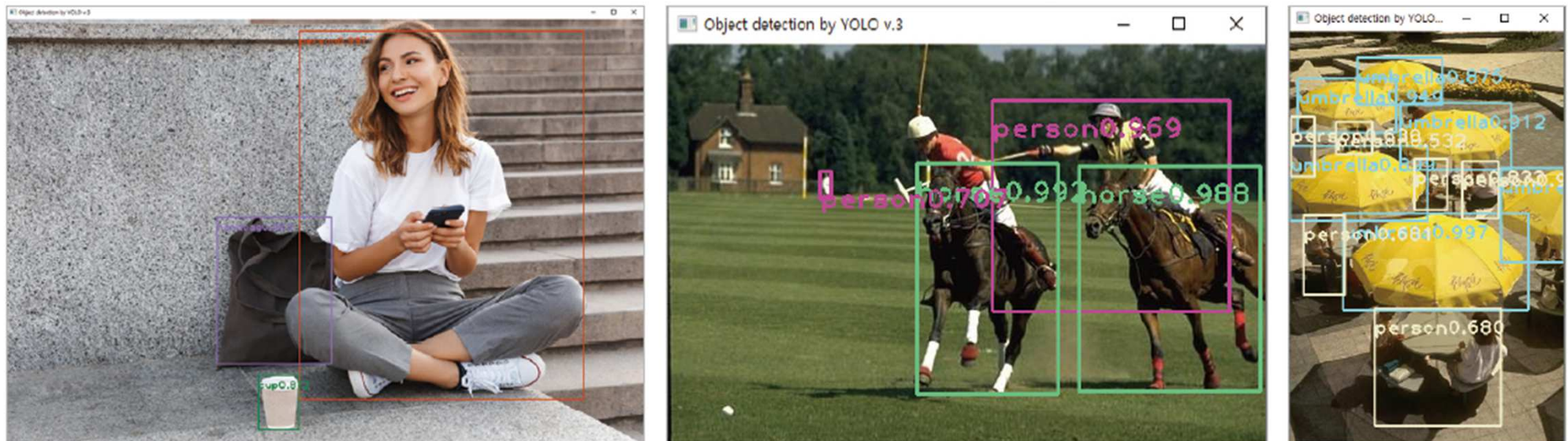


그림 9-27 YOLO v3으로 물체를 검출하는 여러 사례

(실습) YOLO v3으로 동영상에서 물체 검출하기

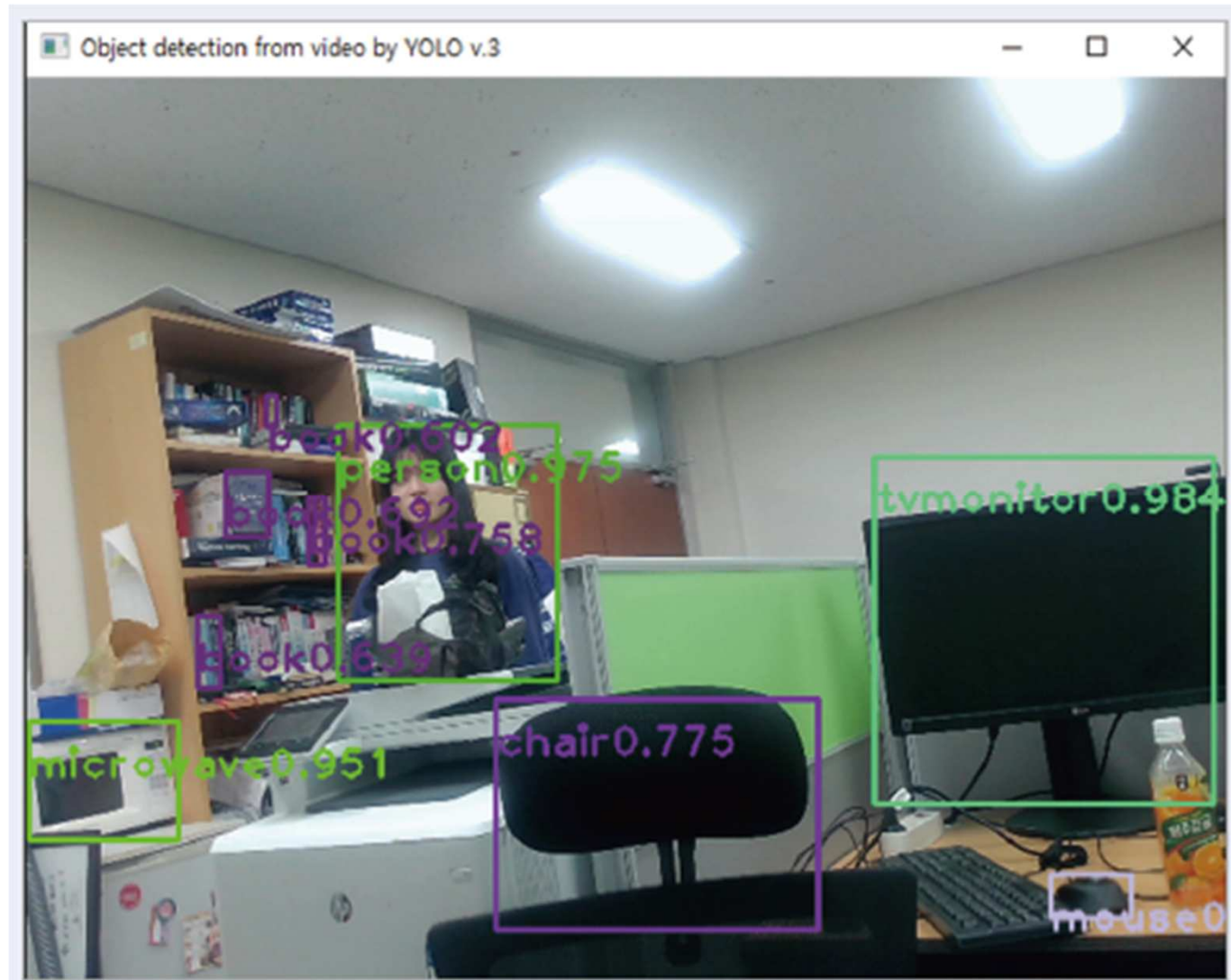
프로그램 9-2

YOLO v3으로 비디오에서 물체 검출하기

1~41행은 [프로그램 9-1]과 같음

```
43 cap=cv.VideoCapture(0,cv.CAP_DSHOW)
44 if not cap.isOpened(): sys.exit('카메라 연결 실패')
45
46 while True:
47     ret,frame=cap.read()
48     if not ret: sys.exit('프레임 획득에 실패하여 루프를 나갑니다.')
49
50     res=yolo_detect(frame,model,out_layers)
51
52     for i in range(len(res)):
53         x1,y1,x2,y2,confidence,id=res[i]
54         text=str(class_names[id])+'.3f'%confidence
55         cv.rectangle(frame,(x1,y1),(x2,y2),colors[id],2)
56         cv.putText(frame,text,(x1,y1+30),cv.FONT_HERSHEY_PLAIN,1.5,colors[id],2)
57
58     cv.imshow("Object detection from video by YOLO v.3",frame)
59
60     key=cv.waitKey(1)
61     if key==ord('q'): break
62
63 cap.release()    # 카메라와 연결을 끊음
64 cv.destroyAllWindows()
```

(실습) YOLO v3으로 동영상에서 물체 검출하기





분할 실습

U-Net

■ U-net

- 의료 영상 분할을 목적으로 개발되어 지금은 자연영상에까지 널리 사용됨
- 샘플링과 업 샘플링을 수행함
- 가장 특이한 점은 지름길 연결(특징 맵을 이어 붙이는 지름길 연결)

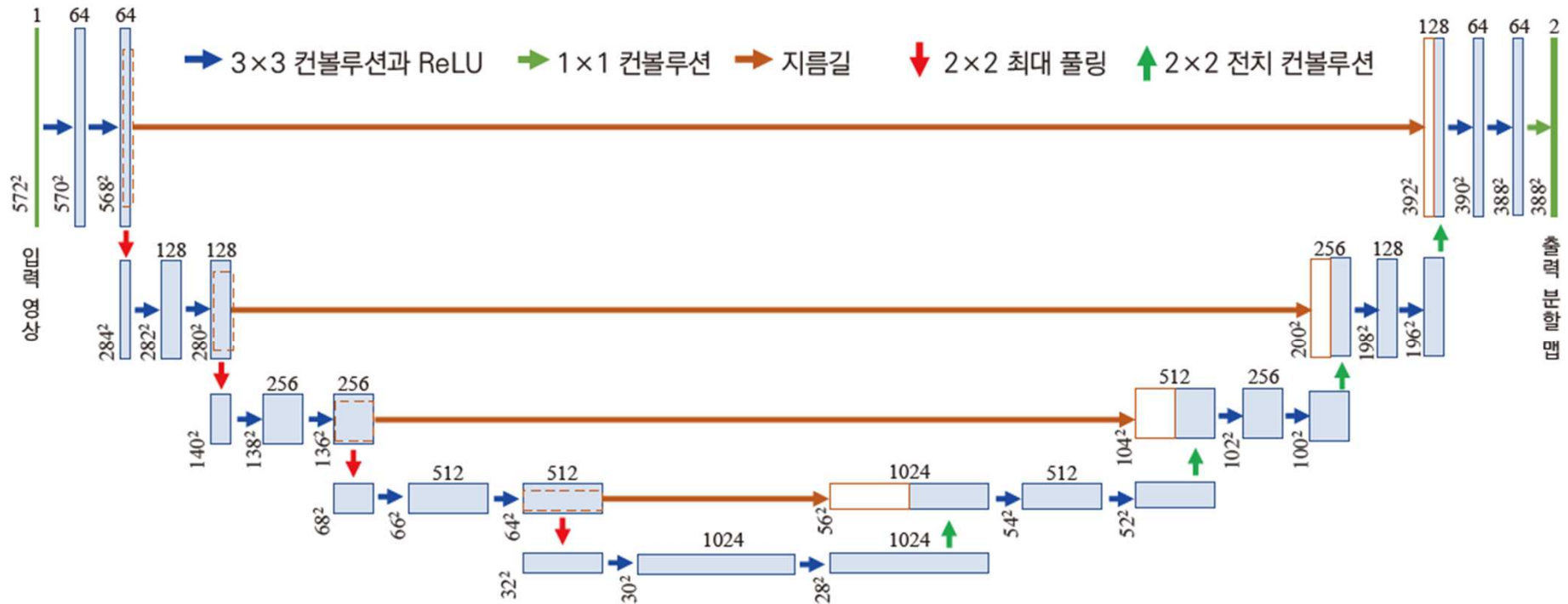


그림 9-36 U-net의 구조

(실습) U-Net을 이용한 분할(semantic segmentation)

■ Oxford IIIT Pet 데이터셋 활용

- 개와 고양이 품종 37개 포함. 품종마다 200여장. 총 7,349장의 영상
- 영상마다 머리 검출을 위한 박스와 분할을 위한 레이블(여기서는 분할 레이블 사용)

<https://www.robots.ox.ac.uk/~vgg/data/pets/>

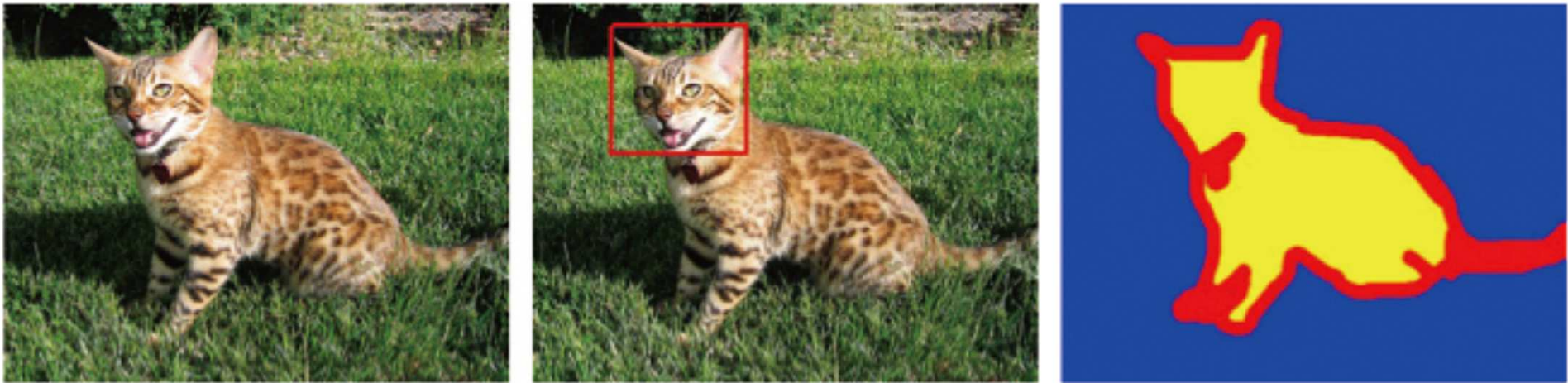


그림 9-39 Oxford IIIT Pet 데이터셋의 예제 영상과 레이블

(실습) U-Net을 이용한 분할(semantic segmentation)

프로그램 9-4

Oxford pets 데이터셋으로 U-net 학습하기

```
01 from tensorflow import keras
02 import numpy as np
03 from tensorflow.keras.preprocessing.image import load_img
04 from tensorflow.keras import layers
05 import os
06 import random
07 import cv2 as cv
08
09 input_dir='./datasets/oxford_pets/images/images/'
10 target_dir='./datasets/oxford_pets/annotations/annotations/trimaps/'
11 img_siz=(160,160)           # 모델에 입력되는 영상 크기
12 n_class=3                  # 분할 레이블 (1:물체, 2:배경, 3:경계)
13 batch_siz=32              # 미니 배치 크기
14
15 img_paths=sorted([os.path.join(input_dir,f) for f in os.listdir(input_
    dir) if f.endswith('.jpg')])
16 label_paths=sorted([os.path.join(target_dir,f) for f in os.listdir(target_
    dir) if f.endswith('.png') and not f.startswith('.')])
17
```

(실습) U-Net을 이용한 분할(semantic segmentation)

```
18 class OxfordPets(keras.utils.Sequence):
19     def __init__(self, batch_size, img_size, img_paths, label_paths):
20         self.batch_size=batch_size
21         self.img_size=img_size
22         self.img_paths=img_paths
23         self.label_paths=label_paths
24
25     def __len__(self):
26         return len(self.label_paths)//self.batch_size
27
28     def __getitem__(self, idx):
29         i=idx*self.batch_size
30         batch_img_paths=self.img_paths[i:i+self.batch_size]
31         batch_label_paths=self.label_paths[i:i+self.batch_size]
32         x=np.zeros((self.batch_size,)+self.img_size+(3,), dtype="float32")
33         for j, path in enumerate(batch_img_paths):
34             img=load_img(path, target_size=self.img_size)
35             x[j]=img
36         y=np.zeros((self.batch_size,)+self.img_size+(1,), dtype="uint8")
37         for j, path in enumerate(batch_label_paths):
38             img=load_img(path, target_size=self.img_size, color_mode="grayscale")
39             y[j]=np.expand_dims(img, 2)
40             y[j]=1 # 부류 번호를 1,2,3에서 0,1,2로 변환
41         return x,y
42
```

(실습) U-Net을 이용한 분할(semantic segmentation)

```
43 def make_model(img_size,num_classes):
44     inputs=keras.Input(shape=img_size+(3,))
45
46     # U-net의 다운 샘플링(축소 경로)
47     x=layers.Conv2D(32,3,strides=2,padding='same')(inputs)
48     x=layers.BatchNormalization()(x)
49     x=layers.Activation('relu')(x)
50     previous_block_activation=x      # 지름길 연결을 위해
51
52     for filters in [64,128,256]:
53         x=layers.Activation('relu')(x)
54         x=layers.SeparableConv2D(filters,3,padding='same')(x)
55         x=layers.BatchNormalization()(x)
56         x=layers.Activation('relu')(x)
57         x=layers.SeparableConv2D(filters,3,padding='same')(x)
58         x=layers.BatchNormalization()(x)
59         x=layers.MaxPooling2D(3,strides=2,padding='same')(x)
60         residual=layers.Conv2D(filters,1,strides=2,padding='same')(previous_
            block_activation)
61         x=layers.add([x,residual])      # 지름길 연결
62         previous_block_activation=x     # 지름길 연결을 위해
```


(실습) U-Net을 이용한 분할(semantic segmentation)

```
63
64     # U-net의 업 샘플링(확대 경로)
65     for filters in [256, 128, 64, 32]:
66         x=layers.Activation('relu')(x)
67         x=layers.Conv2DTranspose(filters,3,padding='same')(x)
68         x=layers.BatchNormalization()(x)
69         x=layers.Activation('relu')(x)
70         x=layers.Conv2DTranspose(filters,3,padding='same')(x)
71         x=layers.BatchNormalization()(x)
72         x=layers.UpSampling2D(2)(x)
73         residual=layers.UpSampling2D(2)(previous_block_activation)
74         residual=layers.Conv2D(filters,1,padding='same')(residual)
75         x=layers.add([x,residual])           # 지름길 연결
76         previous_block_activation=x         # 지름길 연결을 위해
77
78     outputs=layers.Conv2D(num_classes,3,activation='softmax',padding='same')(x)
79     model=keras.Model(inputs, outputs)       # 모델 생성
80     return model
81
```

(실습) U-Net을 이용한 분할(semantic segmentation)

```
82 model=make_model(img_siz,n_class)          # 모델 생성
83
84 random.Random(1).shuffle(img_paths)
85 random.Random(1).shuffle(label_paths)
86 test_samples=int(len(img_paths)*0.1)        # 10%를 테스트 집합으로 사용
87 train_img_paths=img_paths[:-test_samples]
88 train_label_paths=label_paths[:-test_samples]
89 test_img_paths=img_paths[-test_samples:]
90 test_label_paths=label_paths[-test_samples:]
91
```

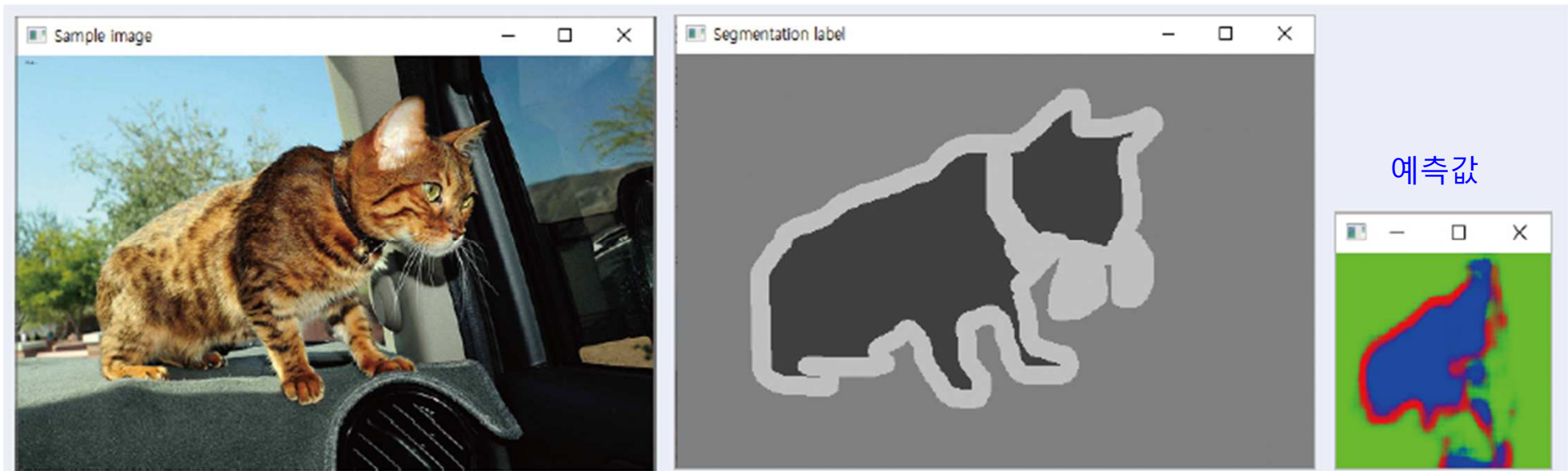
(실습) U-Net을 이용한 분할(semantic segmentation)

```
92 train_gen=OxfordPets(batch_siz,img_siz,train_img_paths,train_label_paths)
    # 훈련 집합
93 test_gen=OxfordPets(batch_siz,img_siz,test_img_paths,test_label_paths)
    # 검증 집합
94
95 model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])
96 cb=[keras.callbacks.ModelCheckpoint('oxford_seg.h5',save_best_only=True)]
    # 학습 결과 자동 저장
97 model.fit(train_gen,epochs=30,validation_data=test_gen,callbacks=cb)
98
99 preds=model.predict(test_gen)
    # 예측
100
101 cv.imshow('Sample image',cv.imread(test_img_paths[0])) # 0번 영상 디스플레이
102 cv.imshow('Segmentation label',cv.imread(test_label_paths[0])*64)
103 cv.imshow('Segmentation prediction',preds[0])
    # 0번 영상 예측 결과 디스플레이
104
105 cv.waitKey()
106 cv.destroyAllWindows()
```


(실습) U-Net을 이용한 분할(semantic segmentation)

입력 영상

참값



(실습) pixellib 라이브러리로 정지 영상 semantic segmentation하기

프로그램 9-5

pixellib 라이브러리로 정지 영상을 의미 분할하기

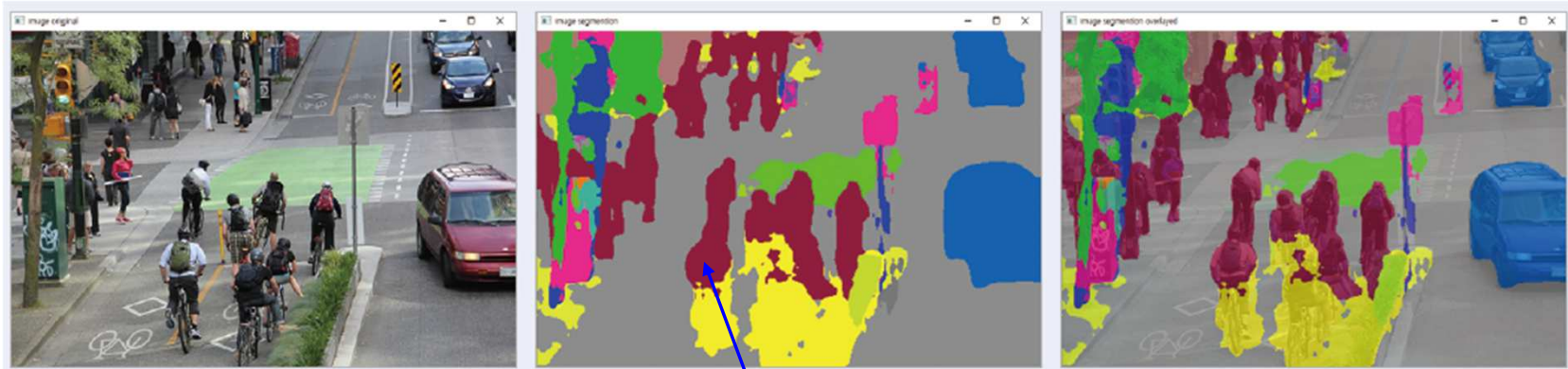
```
01 from pixellib.semantic import semantic_segmentation
02 import cv2 as cv
03
04 seg=semantic_segmentation()
05 seg.load_ade20k_model('deeplabv3_xception65_ade20k.h5')
06
07 img_fname='busy_street.jpg'
08 seg.segmentAsAde20k(img_fname,output_image_name='image_new.jpg')
09 info1,img_segmented1=seg.segmentAsAde20k(img_fname)
10 info2,img_segmented2=seg.segmentAsAde20k(img_fname,overlay=True)
11
12 cv.imshow('Image original',cv.imread(img_fname))
13 cv.imshow('Image segention',img_segmented1)
14 cv.imshow('Image segention overlayed',img_segmented2)
15
16 cv.waitKey()
17 cv.destroyAllWindows()
```

↑
영상에 겹쳐 표시

(실습) pixellib 라이브러리로 정지 영상 semantic segmentation하기

영역만 표시

영역을 영상에 겹쳐 표시



의미 분할이라 사람을
서로 구별하지 않음

분할 결과의 상세 내용

```
In[1]: print(info1)
{'class_ids': [7, 13, 21, 128, 5, 44, 10, 137, 12, 2, 18, 94, 116, 1, 139, 42, 41, 55, 81,
              101, 60, 14, 30],
 'class_names': ['road', 'person', 'car', 'bicycle', 'tree', 'signboard', 'grass',
                 'traffic', 'sidewalk', 'building', 'plant', 'pole', 'bag', 'wall',
                 'ashcan', 'box', 'base', 'runway', 'bus', 'poster', 'stairway', 'earth',
                 'field'],
 'class_colors': [array([140, 140, 140]), array([150, 5, 61]), array([ 0, 102, 200]), ... ..],
 'masks': array([[False, False, False, ..., False], ... ..], 'ratios': [49.06, 15.625, 8.61, ... ..]}
```

(실습) pixellib 라이브러리로 동영상 semantic segmentation하기

프로그램 9-6

pixellib 라이브러리로 비디오를 의미 분할하기

```
01 from pixellib.semantic import semantic_segmentation
02 import cv2 as cv
03
04 cap=cv.VideoCapture(0)
05
06 seg_video=semantic_segmentation()
07 seg_video.load_ade20k_model('deeplabv3_exception65_ade20k.h5')
08
09 seg_video.process_camera_ade20k(cap,overlay=True,frames_per_second=2,output_
video_name='output_video.mp4',show_frames=True,frame_name='Pixellib')
10
11 cap.release()
12 cv.destroyAllWindows()
```

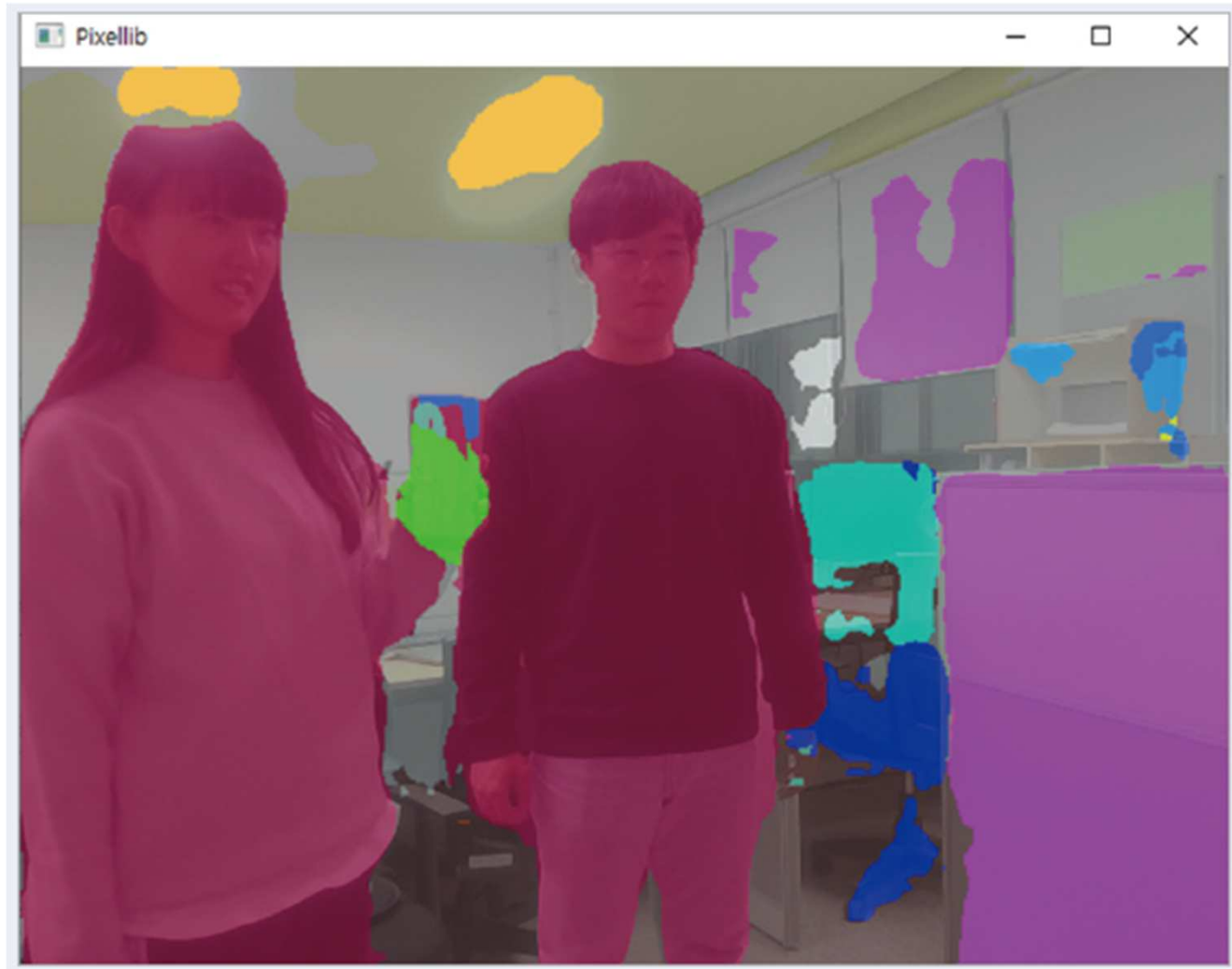
영상에 겹쳐 표시

초당 2프레임 저장

분할 결과를 mp4 파일에 저장

윈도우에 실시간 디스플레이

(실습) pixellib 라이브러리로 동영상 semantic segmentation하기



(실습) pixellib 라이브러리로 정지 영상 instance segmentation하기

프로그램 9-7

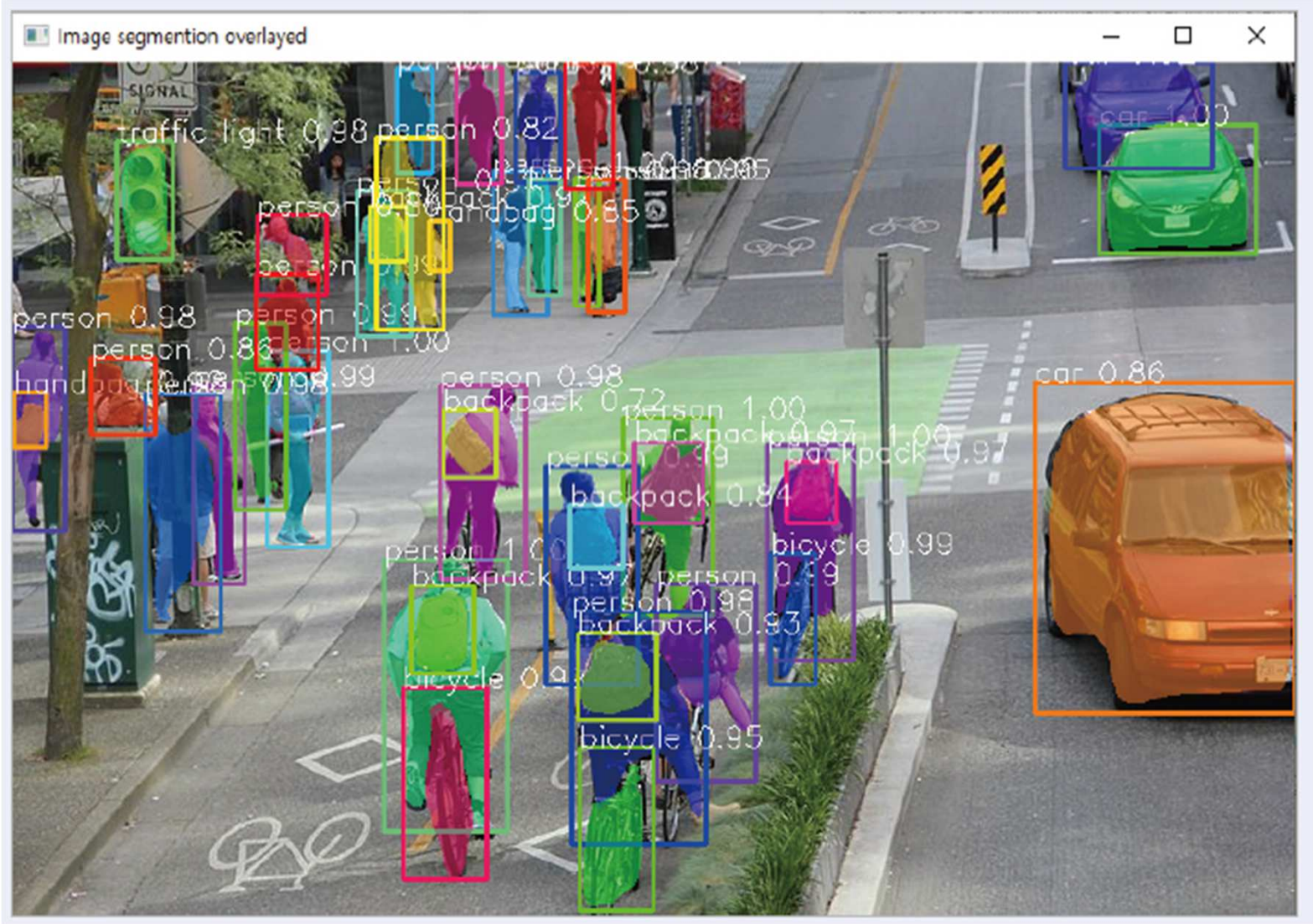
pixellib 라이브러리로 정지 영상을 사례 분할하기

```
01 from pixellib.instance import instance_segmentation
02 import cv2 as cv
03
04 seg=instance_segmentation()
05 seg.load_model("mask_rcnn_coco.h5")
06
07 img_fname='busy_street.jpg'
08 info,img_segmented=seg.segmentImage(img_fname,show_bboxes=True)
09
10 cv.imshow('Image segmentation overlaid',img_segmented)
11
12 cv.waitKey()
13 cv.destroyAllWindows()
```

영역에 박스 씌우고 class 확률 표시

↓

(실습) pixellib 라이브러리로 정지 영상 instance segmentation하기



(실습) pixellib 라이브러리로 동영상 instance segmentation하기

프로그램 9-8

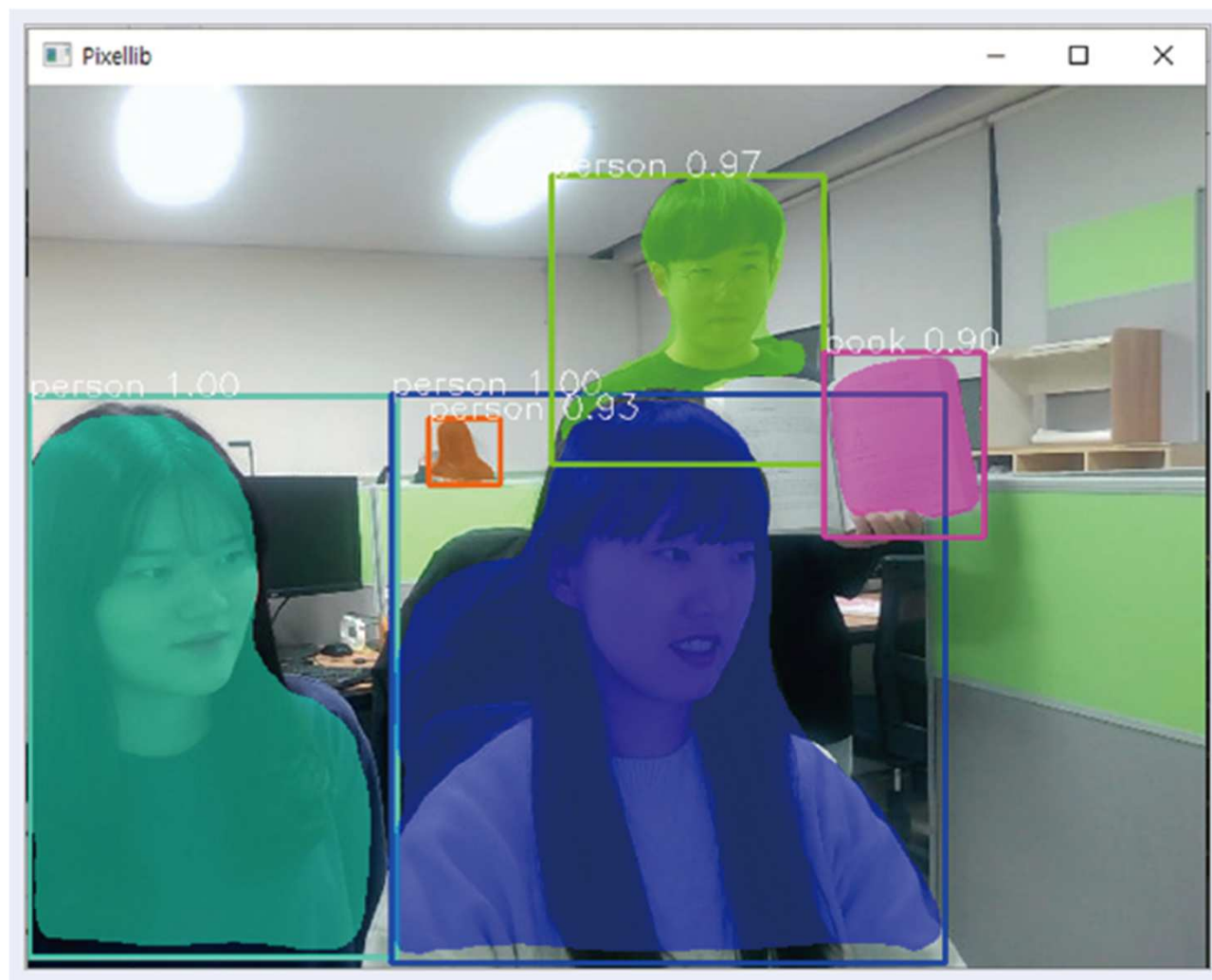
pixellib 라이브러리로 비디오를 사례 분할하기

```
01 from pixellib.instance import instance_segmentation
02 import cv2 as cv
03
04 cap=cv.VideoCapture(0)
05
06 seg_video=instance_segmentation()
07 seg_video.load_model("mask_rcnn_coco.h5")
08
09 target_class=seg_video.select_target_classes(person=True,book=True)
10 seg_video.process_camera(cap,segment_target_classes=target_class,frames_per_
    second=2,show_frames=True,frame_name='Pixellib',show_bboxes=True)
11
12 cap.release()
13 cv.destroyAllWindows()
```

person과 book class를 분할하라는 지시



(실습) pixellib 라이브러리로 동영상 instance segmentation하기



(실습) pixellib 라이브러리를 활용해 배경 바꾸기

프로그램 9-9

pixellib 라이브러리를 활용해 내 맘대로 배경 바꾸기

```
01 import cv2 as cv
02 import numpy as np
03 from PyQt5.QtWidgets import *
04 import sys
05 from pixellib.tune_bg import alter_bg
06
07 class VideoSpecialEffect(QMainWindow):
08     def __init__(self):
09         super().__init__()
10         self.setWindowTitle('배경을 내 맘대로')
11         self.setGeometry(200,200,400,100)
12
13         videoButton=QPushButton('배경 내 맘대로 켜기',self)
14         self.pickCombo=QComboBox(self)
15         self.pickCombo.addItems(['원래 영상','흐릿(조금)','흐릿(중간)','흐릿(많이)',
16                                 '빨강','녹색','파랑'])
17
18         quitButton=QPushButton('나가기',self)
19
20         videoButton.setGeometry(10,10,140,30)
21         self.pickCombo.setGeometry(150,10,110,30)
22         quitButton.setGeometry(280,10,100,30)
23
24         videoButton.clicked.connect(self.videoSpecialEffectFunction)
25         quitButton.clicked.connect(self.quitFunction)
```

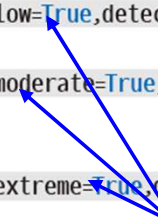
(실습) pixellib 라이브러리를 활용해 배경 바꾸기

```
24
25 def videoSpecialEffectFunction(self):
26     self.cap=cv.VideoCapture(0,cv.CAP_DSHOW)
27     if not self.cap.isOpened(): sys.exit('카메라 연결 실패')
28
29     while True:
30         ret,frame=self.cap.read()
31         if not ret: break
32
33         pick_effect=self.pickCombo.currentIndex()
34         if pick_effect==0:
35             special_img=frame
36         elif pick_effect==1:
37             special_img=change_bg.blur_frame(frame,low=True,detect='person')
38         elif pick_effect==2:
39             special_img=change_bg.blur_frame(frame,moderate=True,detect
40             ='person')
41         elif pick_effect==3:
42             special_img=change_bg.blur_frame(frame,extreme=True,detect
43             ='person')
44         elif pick_effect==4:
45             special_img=change_bg.color_frame(frame,colors=(255,0,0),detect
46             ='person')
47         elif pick_effect==5:
48             special_img=change_bg.color_frame(frame,colors=(0,255,0),detect
49             ='person')
50         elif pick_effect==6:
51             special_img=change_bg.color_frame(frame,colors=(0,0,255),detect
52             ='person')
53
54         cv.imshow('Special effect',special_img)
55         cv.waitKey(1)
```

person 이외를 배경으로 간주



흐릿한 정도



(실습) pixellib 라이브러리를 활용해 배경 바꾸기

```
52     def quitFunction(self):
53         self.cap.release()
54         cv.destroyAllWindows()
55         self.close()
56
57     change_bg=alter_bg(model_type="pb")
58     change_bg.load_pascalvoc_model('xception_pascalvoc.pb')
59
60     app=QApplication(sys.argv)
61     win=VideoSpecialEffect()
62     win.show()
63     app.exec_()
```

